WhisperProtocol: Backend Architecture & Audio Generation

Complete Technical Guide — Acoustic Air-Gapped Data Transmission

Ultrasonic AFSK (18–19 kHz) | X25519 + AES-256-GCM | Reed-Solomon ECC | Half-Duplex Audio Coordinator | JSON-Lines IPC Bridge

1 Core Concept & Architectural Mission

WhisperProtocol is not an AI speech recognition or text-to-speech model. Instead, it is an **acoustic air-gapped data modem**. It encodes digital files, text messages, and cryptographic handshakes into near-ultrasonic sound waves that transmit through the air between computer speakers and microphones without Wi-Fi, Bluetooth, or local networks.

> **Why 18 kHz and 19 kHz?** Human hearing sensitivity drops off sharply above 17 kHz for adults. By transmitting at 18–19 kHz, the audio is completely silent to human ears, but still falls safely within the standard 22.05 kHz Nyquist frequency limit supported by standard 44.1 kHz consumer laptop soundcards and microphones.

2 End-to-End System Architecture

The backend is divided into four distinct layers orchestrated through an IPC bridge:

1. IPC Bridge (`bridge.py`)

Communicates bidirectionally with the Tauri/Rust desktop frontend over standard OS pipes (`stdin` commands, `stdout` events, `stderr` logs).

2. Crypto Engine (`whisper/crypto/`)

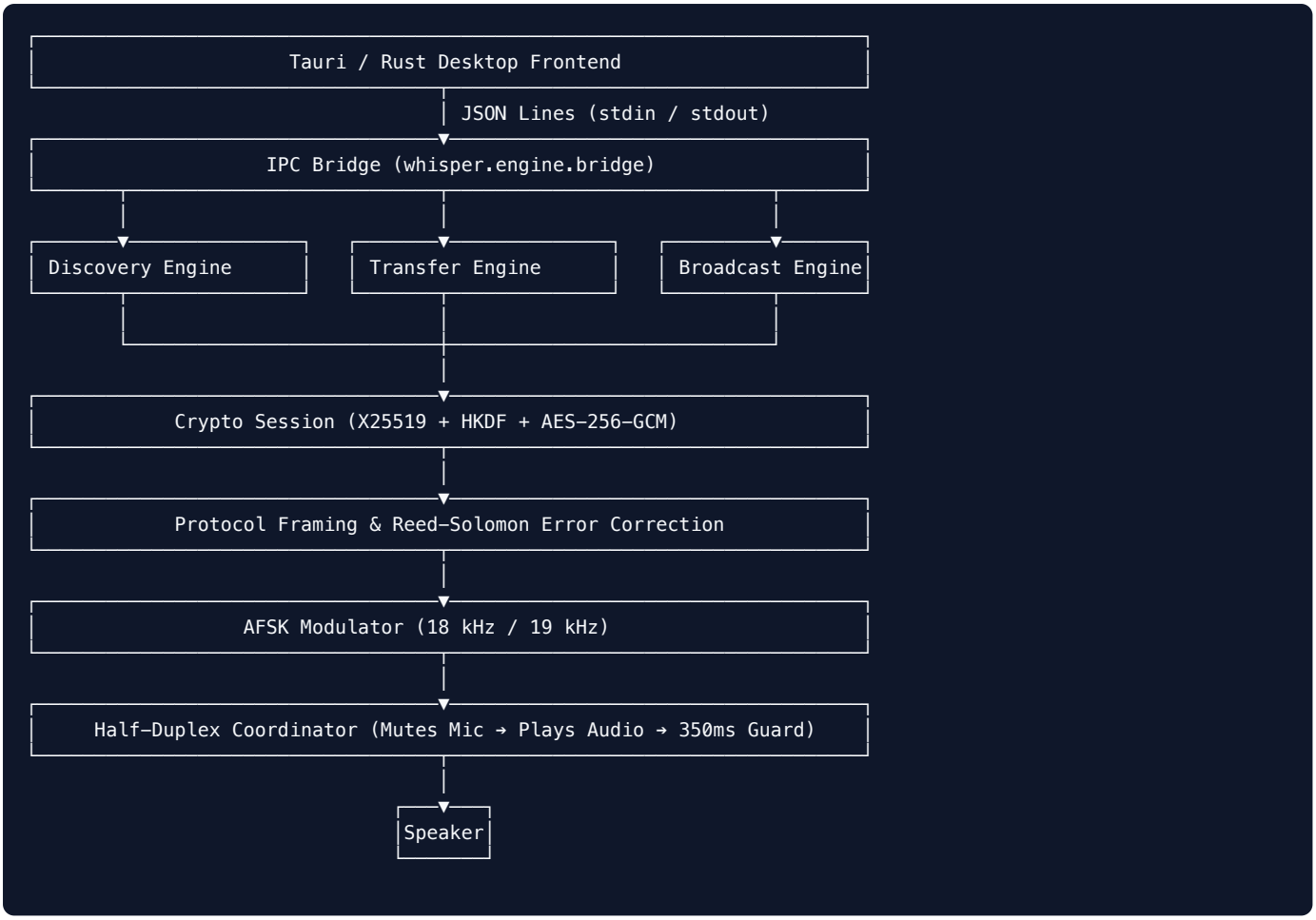
Ephemeral X25519 Diffie-Hellman key exchange, HKDF-SHA256 directional key derivation, AES-256-GCM encryption, and SAS visual authentication.

3. Protocol Engine (`whisper/protocol/`)

Binary frame packing, CRC-16 checksums, Reed-Solomon error correction coding (ECC), and AFSK modulation / demodulation.

4. Audio Coordinator (`coordinator.py`)

Hardware device discovery, half-duplex transmit/receive switching, acoustic decay guard intervals (350 ms), and PortAudio playback.


Tauri / Rust Desktop Frontend
JSON Lines (stdin / stdout)
IPC Bridge (whisper.engine.bridge)
Discovery Engine
Transfer Engine
Broadcast Engine
Crypto Session (X25519 + HKDF + AES-256-GCM)
Protocol Framing & Reed-Solomon Error Correction
AFSK Modulator (18 kHz / 19 kHz)
Half-Duplex Coordinator (Mutes Mic → Plays Audio → 350ms Guard)
Speaker

3 How the Audio is Generated (Deep-Dive Pipeline)

Every message undergoes an 8-stage mathematical and physical transformation before leaving the speaker:

STAGE 1 UTF-8 String & Payload Serialization

User messages or file chunks are serialized into binary bytes. Unicode text is encoded as standard UTF-8.

STAGE 2 Authenticated Cryptographic Encryption

The payload is encrypted with **AES-256-GCM** using an ephemeral session key derived from an X25519 handshake. A unique 12-byte random nonce and a 16-byte authentication tag are prepended to ensure confidentiality, integrity, and replay defense.

STAGE 3 Frame Assembly & CRC-16 Checksum

The bitstream is structured into frames to enable clock synchronization, byte alignment, and data verification:

Field	Size	Pattern / Purpose
Preamble	32 bits	Alternating `1010101010101010...` for receiver AGC & clock synchronization
Sync Word	16 bits	`1111000011110000` marks exact byte alignment boundary
Length	16 bits	Big-endian unsigned integer indicating payload byte length
Payload	Variable	Encrypted message bytes
Checksum	16 bits	CRC-16-CCITT (polynomial `0x1021`) computed over `Length + Payload`

STAGE 4 Reed-Solomon Error Correction (ECC)

Acoustic channels are prone to background noise and multipath room echoes. WhisperProtocol applies **Reed-Solomon RS(255, 223)** forward error correction, adding parity bytes so the receiver can reconstruct lost or corrupted bits without retransmissions.

STAGE 5 AFSK Bit-to-Frequency Mapping

Each bit is mapped to an ultrasonic carrier frequency:

$$\text{Bit 0 (Space)} \rightarrow f_0 = 18{,}000 \text{ Hz}$$
$$\text{Bit 1 (Mark)} \rightarrow f_1 = 19{,}000 \text{ Hz}$$

STAGE 6 Discrete Waveform Synthesis (Math)

Each bit is transmitted as an individual acoustic symbol lasting **20 ms** ($T = 0.02$ s). At sample rate $f_s = 44{,}100$ Hz, the number of samples synthesized per bit is:

$$N_{samples} = f_s \times T_{symbol} = 44{,}100 \times 0.02 = 882 \text{ samples / bit}$$

The discrete time vector t and sine wave $y(t)$ are synthesized via NumPy:

$$t = [0,\ 1/f_s,\ 2/f_s,\ \ldots,\ 881/f_s]$$
$$y(t) = A \cdot \sin(2\pi \cdot f \cdot t) \quad (\text{where } A = 0.8 \text{ peak amplitude})$$

STAGE 7 Anti-Click Windowing (Raised-Cosine Fade)

Abruptly starting or stopping a sine wave produces audible clicks ("spectral splatter"). WhisperProtocol applies a **2 ms raised-cosine fade-in and fade-out** to each symbol:

$$\text{fade_in}(n) = 0.5 \times [1.0 - \cos(\pi \times n / N_{\text{fade}})]$$

The first 88 samples ramp smoothly up from 0.0, and the last 88 samples ramp smoothly down to 0.0.

STAGE 8 Half-Duplex Coordination & Playback

The `HalfDuplexAudioCoordinator` guarantees that the machine never hears its own transmission:

1. **Mute:** Suspends microphone recording and flushes input buffers.
2. **Playback:** Streams the concatenated `float32` array via `sounddevice.play()`.
3. **Decay Guard Interval:** Pauses for **350 ms** while room acoustic reverberations die down.
4. **Re-engage:** Purges residual mic buffers and resumes receiver listening.

4 Protocol Specification & Key Parameters Summary

Parameter	Value	Technical Rationale
Modulation Scheme	AFSK (Audio Frequency Shift Keying)	High noise immunity, robust non-coherent demodulation
Bit 0 Frequency	18,000 Hz	Near-ultrasonic carrier; inaudible to human ears
Bit 1 Frequency	19,000 Hz	1,000 Hz separation ensures clean FFT peak isolation
Audio Sample Rate	44,100 Hz (Standard CD Quality)	Universal cross-platform hardware compatibility
Symbol Duration	20 ms (0.02 seconds)	882 samples/symbol; resilient against multipath delay spread
Baud Rate / Speed	50 symbols / second (50 bps)	Prioritizes acoustic reliability over raw throughput
Fade In / Out	2.0 ms (Raised-Cosine Window)	Eliminates spectral splatter (audible speaker popping)
Peak Amplitude	0.8 (Float32 Range [-1.0, 1.0])	Prevents digital clipping and DAC saturation
Acoustic Decay Guard	350 ms	Prevents room reflections from triggering self-reception
Error Correction	Reed-Solomon RS(255, 223) + CRC-16	Burst noise recovery and guaranteed frame validation
Cryptographic Layer	X25519 + HKDF-SHA256 + AES-256-GCM	Zero-trust air-gapped end-to-end security

5 Reception & Demodulation Pipeline (Sound ➔ Data)

The receiver continuously runs the inverse DSP pipeline on incoming microphone audio:

```
Incoming Audio Chunks (44.1 kHz via PortAudio)
    |
    ▼ [Short-Time FFT / Goertzel Filter]
Energy Extraction: Bin 18 kHz vs. Bin 19 kHz
    |
    ▼ [Signal Detection & Preamble Correlator]
Detects alternating `101010...` preamble pattern
    |
    ▼ [Phase-Locked Clock Recovery]
Aligns 20ms symbol sampling points to the center of energy pulses
    |
    ▼ [Sync Word Boundary Lock]
Locks on `1111000011110000` → Frame start confirmed
    |
    ▼ [Frame Decoding & CRC-16 Validation]
Extracts payload bytes and verifies checksum
    |
    ▼ [Reed-Solomon Error Correction]
Corrects any burst errors or corrupted bits
    |
    ▼ [AES-256-GCM Decryption]
Restores plaintext message or file payload
```

> **Key Takeaway:** WhisperProtocol achieves reliable air-gapped communication by pairing robust digital signal processing (AFSK + raised-cosine filtering + Reed-Solomon ECC) with strict acoustic timing (half-duplex coordinator + 350 ms reverberation decay), creating an invisible and secure sound-based data bridge.